



Informatics Institute of Technology

University of Westminster

COURSE – (LEADING TO)	MSC APPLIED AI
MODULE NAME	FOUNDATIONS OF ARTIFICIAL INTELLIGENCE
MODULE ID	COSC013C
ASSIGNMENT ID	CW1 - PROPOSAL
DESCRIPTION	RESEARCH, JUSTIFY AND IMPLEMENT ONE OR A COMBINATION OF AI TECHNIQUES TO ACHIEVE THE STATED GOAL.
SUBMISSION DATE	2026-08-05
SUBMISSION VIA	THE SUPPORTING MATERIALS ARE ATTACHED AT THE GITHUB REPOSITORY.
MATERIALS INCLUDED	REPORT PROPOSAL NOTEBOOK DEMONSTRATION VIDEO

STUDENT NAME : K.P. SADUSHA NADEESH DE SILVA

UOW ID : 22238997

IIT ID : 20251211

1. Introduction

Most Navigation applications can quickly find the shortest or fastest route between two locations. However, they do not help users choose the best places to stop along the way. Except user add the exact point when creating the route. If a traveler needs to visit an ATM, grab a coffee or stop at a pharmacy, they are often presented with a long list of nearby places without considering which option fits naturally with the journey.

This project proposes an AI Travel Assistant that helps users find the most suitable point of interest (POIs) while travelling. The user provides a starting location, a destination and the type of places the user would like to visit, such as café, ATM, pharmacy or tourist attraction. Instead of simply selecting the nearest option, the solution identifies the locations that best fit the route while considering factors such as travel distance, detour and user preferences.

For this purpose, the solution combines graph search algorithms for route planning, machine learning models for personalized recommendations and optimization techniques for covering multiple stops.

The aim of this project is to demonstrate how different AI techniques can be integrated into a single solution to create smarter travel planning.

2. Problem Statement

Standard GPS and routing systems optimize for a single metric, usually travel time or distance. They do not account for:

- User preferences: a budget traveler and a luxury traveler having fundamentally different priorities when choosing where to stop.
- Environmental context: rush hour traffic, late night safety or rainy weather all affect what a sensible recommendation looks like.
- Stop ordering: The order in which the user visits when there are multiple stops and the travel distance and time to cover them.

3. Proposed solution design

The proposed system is implemented as a Jupyter Notebook using python. It follows a step-by-step pipeline that transforms a user's input into a complete travel itinerary with a map. The pipeline consists of five main stages.

Stage 1: Road network loading

The road network for the selected city is downloaded from OpenStreetMap using the OSMnx library and stored locally as a GraphML file. Each road segment contains information such as travel distance and travel time which are used during route planning.

Stage 2: Route Search

The user provides a starting location and destination, which are converted into coordinates through geocoding. The system then uses search algorithms to calculate the optimal route along with performance and resource consumption in mind.

Stage 3: Point of Interest (POI) Selection

The user specifies the types of places they would like to visit during the journey, such as an ATM, café or pharmacy. Matching points of interest are retrieved from OpenStreetMap and filtered to include only those located close to the planned route. This ensures that recommended stops require only a small detour rather than taking the user far from their journey.

Stage 4: Personalized Recommendation

The filtered POIs are ranked using machine learning models trained for different user profiles. In this project, two profiles are considered. A **Budget traveler**, who prefers locations that require minimal detours, and a **Luxury Traveler**, who considered high rated places. The models are trained using synthetic data and then used to predict the most suitable POI for each requested category.

Stage 5: Places Optimization

After selecting the best POI for each category, system determines the most efficient order in which to visit them. This was done using a **Travelling Salesperson Problem (TSP)**, to evaluate the best overall order of stops.

4. AI Techniques used

This project integrates three core areas of AI. Search algorithms, knowledge representation and machine learning for each of the key roles.

Search algorithms: Graph search algorithms are used to find the optimal route between locations. A* will be the main algorithm used.

Knowledge Representation: Used to model simple real-world rules that affect travel decisions. This was handled using predefined rules to adjust the suitability of candidate POIs.

Machine Learning: Used to rank candidate points of interest according to different user preferences. The solution uses Decision Tree regressor models trained with synthetic data for two user profiles.

5. Criteria checked when implementing the solution

The project uses multiple checks to check whether the solution works correctly.

- Search Efficiency: A* should explore fewer graph nodes when finding a route

- ML Accuracy: Decision Tree Regressor models should achieve an R^2 score of at least 0.80 on test data.
- POI Coverage: The system should find at least one suitable point of interest (POI) for each category requested by the user along the route area.
- Profile difference: Budget and luxury routes should create different routes from the same POI options.

Folium maps will be created with the route and POI. Included automated tests to verify the solution works as expected.

6. Ethical considerations and responsible use

The project is an academic prototype designed for demonstration purposes. It does not book services or share user location data.

All maps and POI data come from OpenStreetMap which provides open-source data under its license. The ratings used for the ML model are artificially created for testing and marked as synthetic.

7. Conclusion

The solution shows how different AI techniques can work together in one application for a user requirement.